

Saltwyck Market Penetration Test Report

Sep 30, 2026 · @Jeremie

Target: <https://xd3fce742a76831e.labs.penone.ai> (Saltwyck Market, a TryHackMe CTF lab).

Authorization: TryHackMe Live Course, explicit user authorization in chat. 2 of 12 CTF flags confirmed.

Executive summary

2 of the 12 flags in this CTF-style lab were confirmed, both in the same area: the order-return (refund) and checkout/promo-code logic are missing concurrency and state guards, letting an ordinary shopper account extract more money back than it ever paid. A third real bug (checkout never clears the basket, and the stock check has the same race-condition class, allowing a single-unit item to be oversold) was confirmed reproducible but never produced a flag string of its own.

Everything else tested — session/JWT handling, an internal GraphQL API discovered via a deprecated route, mass assignment, every stored/reflected output point found, SQL/NoSQL injection, path traversal, cross-account object access (BOLA) on every reachable object type, function-level access control on seller/staff/admin routes, CORS, password reset, login rate limiting, and several access-control-bypass header tricks — came back clean. No SSRF-capable field was found anywhere, and no route into a seller or staff identity was found, which blocks testing the categories that need one.

A gap in this project's own evidence-logging tooling was also found and is reported separately, at the user's request to test and improve the harness itself, not just the target.

Update after a second testing pass: a further, severe access-control bug was found (Finding 4 below) — any shopper can read any other shopper's support tickets by guessing their order reference, with no ownership check at all. Two more real findings (session/logout handling, and a GraphQL resource-consumption gap) were also confirmed, neither carrying a flag string. Flag count remains 2 of 12; these are additional confirmed vulnerabilities worth fixing regardless.

Finding 1 — Refund basis / missing idempotency on returns — CRITICAL

Flag: `SWK{refund-basis-3626c224b4248641}`

POST /account/orders/{ref}/return (parameter: item_id , the caller's own order line) has two independent bugs that both trip the same server-side ledger check and the same flag:

- **No idempotency guard.** Calling it twice in a row with the same item_id , on an order the caller genuinely owns and already returned once, refunds it a second time.
- **Wrong refund basis.** If the order had a promo-code discount applied, a single (non-duplicate) return refunds the pre-discount subtotal rather than the amount actually paid — over-refunding by exactly the discount amount on the very first call.

Either bug alone makes order_refunded_total exceed order_paid , and the server detects this itself and returns it in the response.

Proof (reproduced independently 4 times, 3 accounts, both trigger paths):

Order	Path	Response
SWK-2026-000701	Duplicate call, no discount	1st: refunded_cents:11602, order_refunded_total:11602, order_paid:11602 (correct). 2nd (same item_id): order_refunded_total:23204, order_paid:11602, ledger_exception:"SWK{refund-basis-3626c224b4248641}"
SWK-2026-000712	Duplicate call, no discount (clean-state repro)	Same pattern, same flag. Evidence ev_000147 / ev_000148 .
SWK-2026-000717	Single call, discount applied	Subtotal \$220.90, THANKYOU discount -\$15.00, paid \$205.90. One return call: refunded_cents:22090, order_refunded_total:22090, order_paid:20590, ledger_exception:"SWK{...}" — over-refunded by exactly the discount, on the first call.

Cross-account / cross-order misuse of item_id was explicitly tested and is correctly blocked (404 no such line for a foreign item_id on your own order, 404 no such order for someone else's order) — this is narrowly a missing already-refunded / wrong-basis check, not a broader IDOR.

Impact: any shopper can generate unlimited store credit against their own real orders, no elevated privilege needed. One test account reached \$787.36 of credit against \$98.42 ever paid, via 8 concurrent duplicate-return calls. Whether that credit is currently spendable anywhere was not conclusively established.

Remediation: compute the refund as `paid - already_refunded` for that specific order line, inside a transaction/row-lock keyed on the line, and reject a second return once the line's refundable balance is zero.

Finding 2 — Promo-code redemption race condition — HIGH

Flag: `SWK{promo-race-2e7ce17a10edc58b}`

The one-time-use promo code `THANKYOU` (a flat \$15.00 discount, found by guessing a short list of plausible marketing codes — 11 other guesses failed) is correctly rejected on a second *sequential* use (`400 you have already used that code`). Firing two `POST /checkout` requests **concurrently**, from the same account and cart, both carrying `promo_code=THANKYOU`, results in **both succeeding** with the discount applied to both — a classic check-then-act race, with no row lock around the redemption check.

Proof: fresh account, single item in cart, two backgrounded `curl` calls fired in the same shell line.

Order	Result
SWK-2026-000718	<code>ok:true, discount_cents:1500</code>
SWK-2026-000719 (lost the race)	<code>ok:true, discount_cents:1500, reconciliation_note:"SWK{promo-race-2e7ce17a10edc58b}"</code>

Impact: any shopper can multiply a one-time discount code by the number of concurrent requests fired — repeatable revenue loss on every future discount code the business issues, not just this one.

Remediation: make "mark code used for this account" and "apply discount" one atomic operation (a unique constraint on `(account_id, promo_code)`, checked before the order is finalised).

Finding 3 — Checkout never clears the basket; stock check races the same way — confirmed, no flag observed

`POST /checkout` never empties the basket on success — calling it again immediately afterward (no race needed, plain sequential re-submission) creates another full, separately-paid order from the identical cart contents, indefinitely. Confirmed 3 times in a row from one cart.

Firing several of these concurrently across different accounts against the **same single-stock product** oversold it: in one run, 3 of 6 concurrent buyers for a stock=1 item all got a confirmed order; in a second run against a stock=2 item with 8 concurrent buyers, exactly 2 succeeded — the window is timing-dependent, as expected for a genuine TOCTOU race rather than a deterministic bug.

No `ledger_exception`-equivalent flag field was ever observed on a checkout response under either the duplication or the oversell condition, across this session's own testing and a dispatched subagent's independent run. Reported as a confirmed, high-severity, reproducible finding without a flag credit.

Remediation: clear or expire the basket transactionally as part of a successful checkout; decrement and check stock inside the same transaction/row-lock as the order creation, not as a prior separate read.

Finding 4 — Broken object-level access control on support-ticket lookup by order reference — CRITICAL

What it is: `GET /support/lookup?q={order_ref}` (e.g. `q=SWK-2026-000701`) has **no ownership check at all**. Any authenticated shopper can read every support ticket ever filed about an order they do not own — full subject, thread body, and author name — just by supplying that order's reference. This is inconsistent with the SAME endpoint's other lookup mode: `q={TK-number}` (a ticket's own case number) IS correctly scoped to the ticket's creator. The bug is specific to the order-reference branch of the lookup.

How it was proven: using `bob`'s own session (no relation to the order), `GET /support/lookup?q=SWK-2026-000701` (an order that belongs to a different account, `alice`) returned all 7 of `alice`'s real tickets (TK-4097, TK-4098, TK-4100 through TK-4105) with full subject/body/author content rendered into `bob`'s page. Cross-checked that partial/wildcard values do not leak further (`q=SWK-2026-0007` and `q=SWK` both correctly return "No case is open" — exact match only, no SQL-LIKE-style amplification).

Impact: order references are sequential and easily enumerable (this session alone created refs spanning roughly SWK-2026-000700-000726). Any authenticated user — including a brand-new, zero-privilege shopper account — can walk that sequence and read every other user’s support-ticket correspondence on a shared lab instance with dozens of concurrent students, with no rate limit on this specific lookup observed.

Remediation: the order-reference lookup branch must filter by the querying account’s own order ownership, exactly as the ticket-number branch already does.

Finding 5 — Logout does not invalidate the session — MEDIUM

The `swk_session` cookie is a pure stateless JWT with no server-side revocation list. A token captured or saved before `GET /logout` continues to authenticate successfully afterward (verified: replaying a saved token returned `200` on `/account` post-logout). A leaked or intercepted token remains valid for its full ~12-hour lifetime regardless of the user logging out. **Remediation:** maintain a minimal server-side revocation/blacklist (or a token version/generation number bumped on logout) checked alongside signature validation.

Finding 6 — No query-cost limiting on GraphQL — MEDIUM

`/graphql` has introspection disabled but no query-cost, depth, or batching limit of any kind. A single request with 150 aliased `product(id)` lookups executed in 0.56 seconds with a full response and no throttling. This is an OWASP API4:2023 (Unrestricted Resource Consumption) gap; not scaled further in testing out of care for the shared course infrastructure. **Remediation:** add a query-cost analyzer or a hard cap on aliases/fields per request.

Coverage — tested and confirmed clean

Area	Result
Session / JWT	<code>swk_session</code> is RS256-signed (HttpOnly, Secure, SameSite=Lax). <code>alg:none</code> forgery and a <code>kid</code> - path-injection/HMAC-key-confusion attempt were both rejected.

Area	Result
Login controls	Rate-limited at 429 after 8 failed attempts, correctly per-account not per-IP. Password reset gives an identical response for valid/invalid email (no enumeration). The <code>next</code> redirect parameter rejects any non-relative value (no open redirect).
Mass assignment	Registration and <code>PATCH /api/account</code> only accept <code>first_name</code> / <code>last_name</code> / <code>phone</code> / <code>preferences</code> — <code>role</code> , <code>seller_id</code> , <code>entitlements</code> , <code>email</code> , <code>password</code> are all rejected.
XSS	Every text field with a rendering context found (account name, preferences incl. an SSTI probe <code>{{7*7}}</code>), support-ticket fields, product Q&A, search box) HTML-escapes correctly. No execution observed anywhere.
Injection	Boolean-based differential SQL injection and a NoSQL-operator-style JSON login payload both came back clean.
BOLA	Orders, order documents, and returns are correctly scoped per-owner (404, not 403 — no existence leak). Support-ticket viewing is correctly scoped to the creator; ticket <i>creation</i> accepts an unvalidated <code>order_ref</code> (minor data-integrity gap, no data exposure observed).
BFLA / access control	<code>/seller</code> , <code>/staff</code> , <code>/admin</code> all return a clean 403 for a shopper session. Header tricks (<code>X-Forwarded-For</code> , <code>X-Original-URL</code> , <code>X-Rewrite-URL</code>) and path-case/trailing-slash variants had no effect. No self-service seller-onboarding route and no default staff credentials were found.
Path traversal	The order-document endpoint validates against a real per-order document list server-side, not a filename pattern.
CORS	No <code>Access-Control-*</code> headers anywhere, including the JSON API and GraphQL.

Area	Result
Shadow API	A retired <code>/api/v1/*</code> surface returns <code>410 Gone</code> pointing to <code>/graphql</code> — an undocumented GraphQL API linked from nowhere in the UI. Introspection is disabled but "did you mean" validation errors leak field/type names; mapped to 5 read-only root fields (<code>products</code> , <code>product</code> , <code>seller</code> , <code>questions</code> , <code>categories</code>), none exposing data a shopper couldn't already see via the normal web app.

Gaps: no seller or staff identity was ever obtained, which blocks testing categories that need a second, different-privilege identity (the product-question moderation queue, in particular, could not be observed at all). Whichever of the remaining 10 flags live behind a seller/staff view are, by construction, untested.

Harness finding — evidence-logging hook is silently bypassed by `cd`

Reported because the user asked for the harness itself to be tested and improved, separately from the target application.

`.claude/hooks/evidence-log.js` resolves which engagement to log against via `process.cwd()` — the Bash tool's own persisted shell working directory, not a path fixed to the project root. The moment an agent's shell `cd`s into any directory that lacks `CURRENT_ENGAGEMENT.md` (this session: the engagement folder itself, and a scratch directory used for `curl` cookie jars), the hook's file read throws, it silently exits, and **every subsequent Bash call is dropped from the evidence log with no error surfaced.**

Confirmed empirically: evidence logging stopped completely for roughly 60 consecutive `curl`-based calls — spanning both confirmed flags' original discovery — while the shell's `cwd` was inside a scratch directory, and resumed the instant the `cwd` returned to the project root. Both flags were then independently re-reproduced from a clean state specifically to obtain real, verified evidence ids.

This is the same category of gap as two others already on record in this project for its browser-automation tools, but more consequential, since it affects plain Bash + `curl` — the tool this project's own documentation recommends for exactly this kind of API-level testing.

Fix: resolve the project root from a fixed, absolute location (the hook's own file path, or an environment variable set once by the harness runner) instead of trusting the calling shell's current directory.

Minor findings (no flag, lower severity)

- Missing `Content-Type` header on `POST /account/orders/{ref}/return` (even with a valid `item_id`) triggers a generic HTTP 500 — unhandled-exception path, no stack trace leaked, but a second distinct trigger for the same class already known from a non-numeric `item_id`.
- `Content-Security-Policy` allows `style-src 'unsafe-inline'` (the rest of the CSP is strict and good) — CSS-injection-only gap.
- HSTS is present (`max-age=31536000`) but missing `includeSubDomains / preload`.
- Plus-addressing (`name+tag@example.com`) is not normalized for duplicate-account detection at registration, letting one real mailbox register multiple distinct shopper accounts. Minor account-confusion risk only.
- Security-logging failure (OWASP A09), confirmed directly: the account page's "Security activity" section shows "Nothing recorded" even immediately after deliberately triggering the known double-refund `ledger_exception` on that same account — the server detects and flags the invariant violation in its API response but never surfaces it as a visible security event.
- Source code for Saltwyck Market is definitively confirmed **not accessible anywhere** (exhaustive checks against both the live site and the local project filesystem). Tech fingerprint from headers alone (no source): Node.js / Express, version headers deliberately stripped otherwise.
- A deeper prototype-pollution attempt (nested `__proto__` one level inside the `preferences` object, and `__proto__` inside an array element) was also tested and came back clean — no pollution observed. An unauthenticated GraphQL root field ``platformDigest { feeBps, sellerCount }`` was also found (feeBps:1450 matches the "sellers keep 85.5%" footer exactly) — a minor schema-surface finding, not sensitive data. The ticket-lookup BOLA (Finding 4) has no rate limit either: 18 tickets planted against one `order_ref` in a loop with zero throttling, and no other ticket-access route exists beyond the one ``q=`` lookup.
- **Logout does not invalidate the session server-side.** A JWT saved before calling `GET /logout`, then replayed verbatim afterward, still authenticates (`200` on `/account`). The session is a pure stateless JWT with no revocation/blacklist — a leaked or

captured token remains valid until its natural ~12h expiry regardless of logout. OWASP A07.

- **GraphQL has no query-cost, depth, or batching limit.** A single request with 150 aliased `product(id)` lookups executed in 0.56s with a full response and no throttling or rejection. OWASP API4:2023 (Unrestricted Resource Consumption). Not scaled further given the shared course backend.

Checklist coverage confirmed clean (no new issues)

Email normalization (case-insensitive on both registration-duplicate-detection and login, correctly), JSON-body login (same security posture as form-encoded), no alternate auth endpoints, no duplicate-account race on registration (2 concurrent same-email registrations → exactly one succeeds), HTTP parameter pollution on cart/checkout (first value always wins, no bypass), return endpoint operates per order-line not per-unit (no over-return-quantity case exists), no order cancel/ship/status transition endpoints exist beyond return, no WebSocket/SSE/Swagger/debug endpoints beyond a standard `/healthz`, Host-header/X-Forwarded-Host on password reset accepted without observable differential (full confirmation would need email visibility, which isn't available).

Recommendations, in priority order

1. Fix the return/refund basis and idempotency bug (Finding 1) — compute refunds from net-paid, enforce "at most once" per order line, inside one transaction.
2. Fix the promo-code and stock-check race conditions (Findings 2 and 3) — both need an atomic check-and-mark instead of separate read/write steps.
3. Make checkout clear or expire the basket transactionally on success.
4. Validate that `order_ref` on ticket creation actually belongs to the calling account.
5. Fix the evidence-logging hook's working-directory dependency described above.